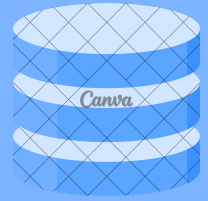




DOCUMENTOS MÉDICOS BIBLIOTECA NACIONAL DE MEDICINA DE LOS ESTADOS UNIDOS

- **Andrés Santos Tovar**
- **David Sánchez**
- **Leonardo Becerra**
- **Maria Paula Ramos**

Entendimiento del Negocio (Clasificación Automática de Artículos Científicos en PubMed mediante Graph Machine Learning)



- Biomédica > 30M artículos biomedicos(medicina, genética, farmacología, etc)
- **Nodos:** Artículos, autores, términos MeSH
- **Relaciones:** Citas (dirigidas), colaboraciones o términos compartidos (no dirigidas).

- **MeSH(Medical Subject Headings):** Tesauro jerarquico con 16 categorías principales(enfermedades, tratamiento, diagnósticos, fármacos)
- **Usado para clasificación temática, interoperabilidad entre base de datos, identificación de tendencias de investigación.**



- **Detección de efectos adversos de medicamentos**
- **Identificación de patrones genéticos en tratamientos**
- **Monitoreo de brotes epidemiológicos mediante análisis semántico**

- **Automatizar la categorización de nuevos artículos**
- **Optimizar búsquedas bibliográficos y descubrimiento de conocimiento.**
- **Extraer insights de relaciones ocultas en la red de citas y términos MeSH.**

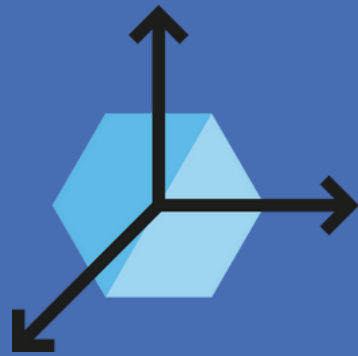


Desarrollar varios modelos de aprendizaje automático sobre grafos (Graph Machine Learning), capaces de predecir las categorías MeSH correspondientes a artículos científicos nuevos. Estos modelos se basarán principalmente en la estructura de citas entre artículos y en sus relaciones semánticas, aprovechando la riqueza estructural y conceptual de la base de datos PubMed.



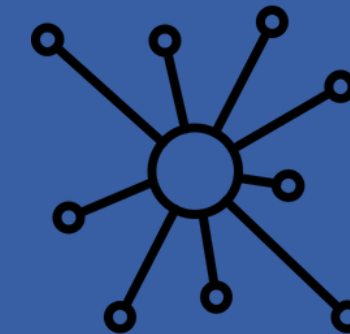
Objetivos de minería de datos

- Analizar la estructura de la red de artículos, identificando nodos clave y patrones para entender cómo se distribuyen las categorías MeSH.



- Usar técnicas como Word2Vec, BERT y Node2Vec para convertir artículos y sus relaciones en representaciones vectoriales. text

- Implementar modelos GGN para predecir múltiples etiquetas MeSH por artículo (clasificación multilabel).

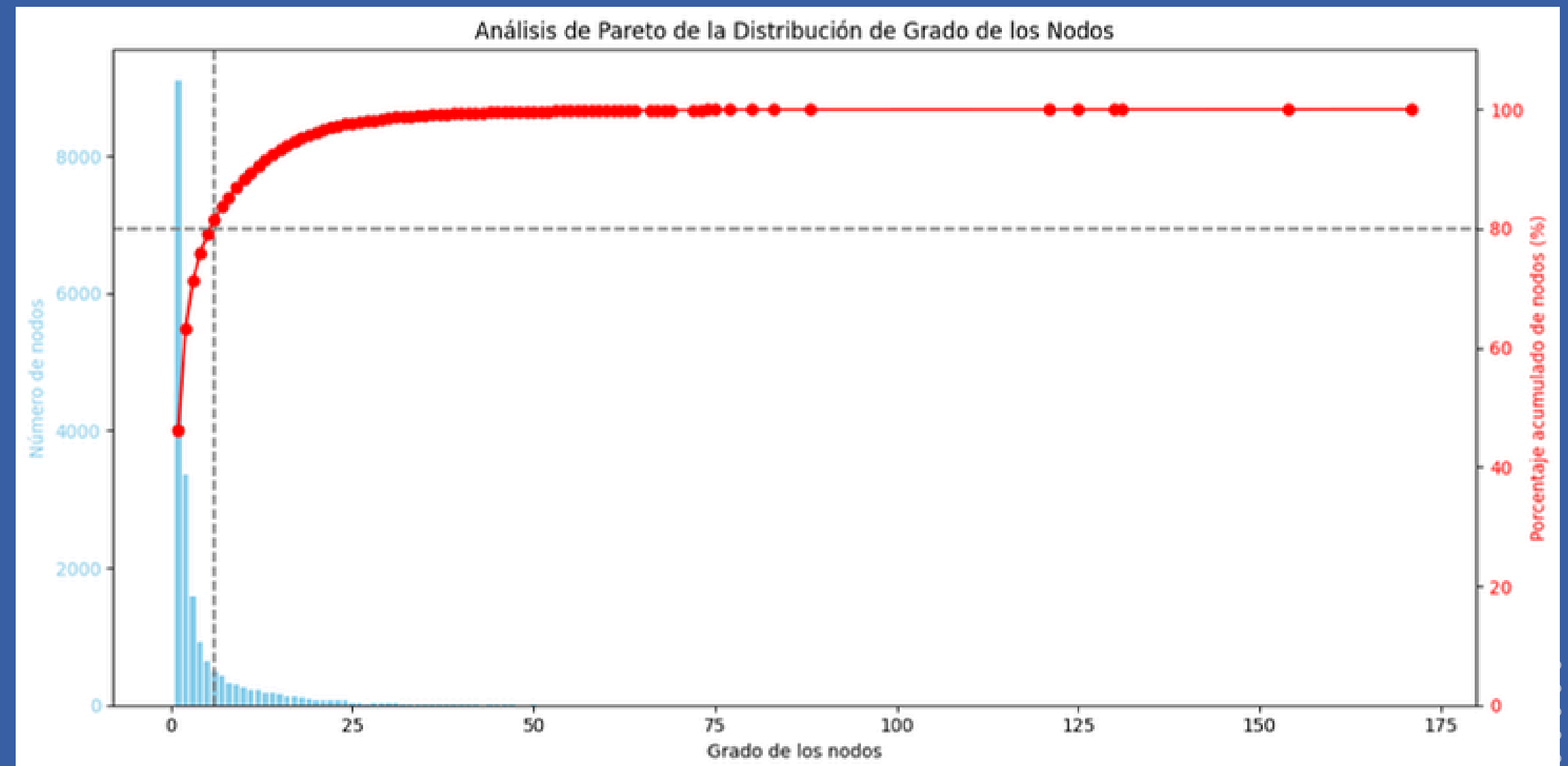


- Medir el rendimiento con métricas como Accuracy y ROC-AUC, seleccionando los modelos más precisos y generalizables.



- Número de nodos: 19717
- Número de conexiones 44324
- Promedio de grado: 4.4960
- Nodo con el grado más alto: 11450 con grado 171
- Número de triángulos en el grafo: 12520

Descripción de los datos



Tenemos los resúmenes de cada artículo médico transformado en un vector TF-IDF de tamaño 500 para cada uno.

Análisis de las features

Principales índices relevantes

Clase 0

- Índice: 0, Peso sumado: 278.2930
- Índice: 15, Peso sumado: 104.4729
- Índice: 285, Peso sumado: 89.2431
- Índice: 32, Peso sumado: 88.5773
- Índice: 8, Peso sumado: 88.2313

Clase 1

- Índice: 149, Peso sumado: 149.0848
- Índice: 89, Peso sumado: 137.0595
- Índice: 162, Peso sumado: 130.8062
- Índice: 16, Peso sumado: 127.3563
- Índice: 346, Peso sumado: 124.0419

Clase 2

- Índice: 196, Peso sumado: 188.9160
- Índice: 449, Peso sumado: 181.0965
- Índice: 477, Peso sumado: 145.0985
- Índice: 89, Peso sumado: 144.5890
- Índice: 444, Peso sumado: 131.7064

Análisis de las features

Indices que mas se repiten

- Índice 162: aparece en el top de 3 clases
- Índice 386: aparece en el top de 2 clases
- Índice 15: aparece en el top de 2 clases
- Índice 196: aparece en el top de 2 clases
- Índice 139: aparece en el top de 2 clases

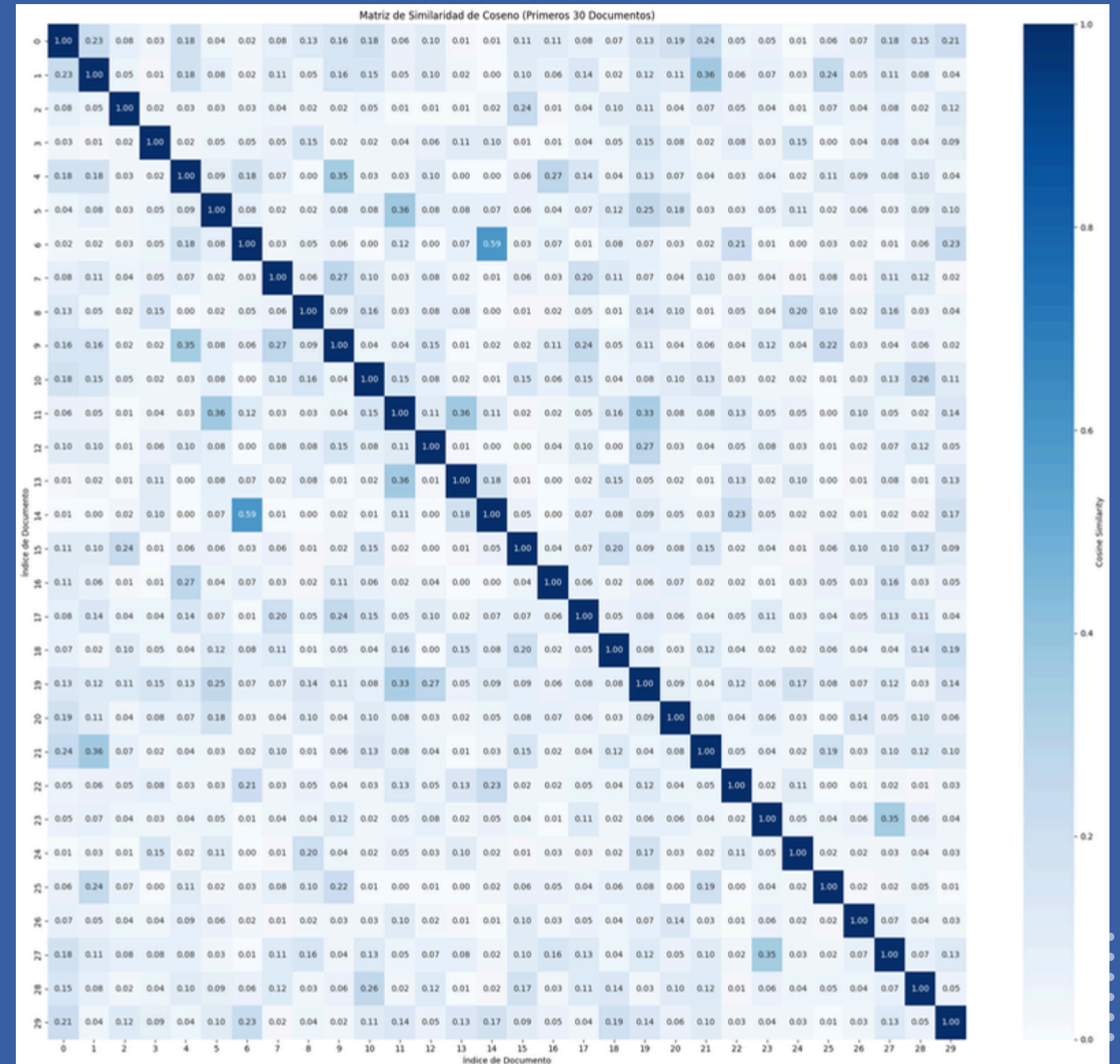
Similaridad con doc 100

- Documento 13714: Similaridad = 0.641
- Documento 13517: Similaridad = 0.625

Similaridad con doc 50

- Documento 7259: Similaridad = 0.693
- Documento 18136: Similaridad = 0.682

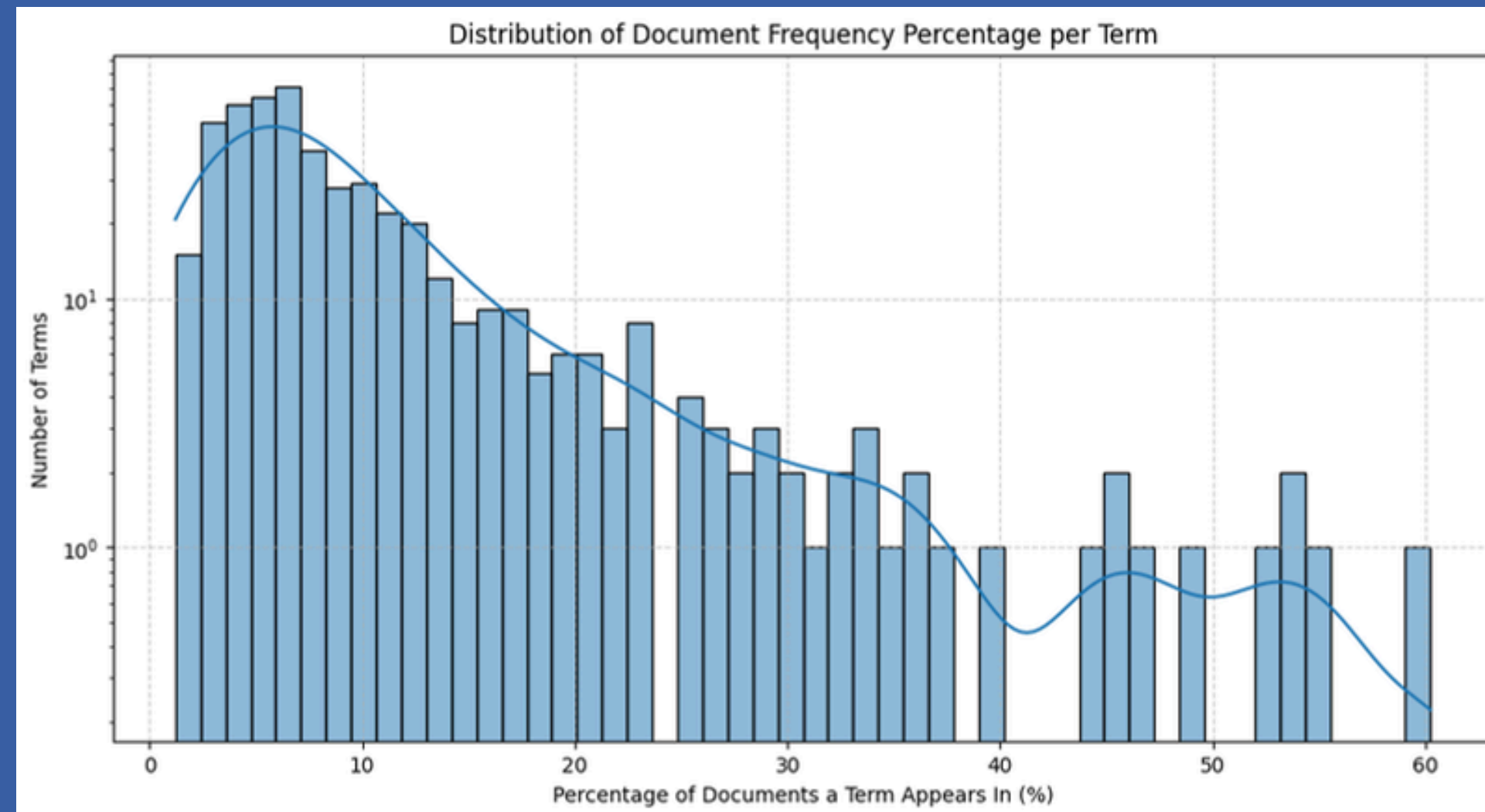
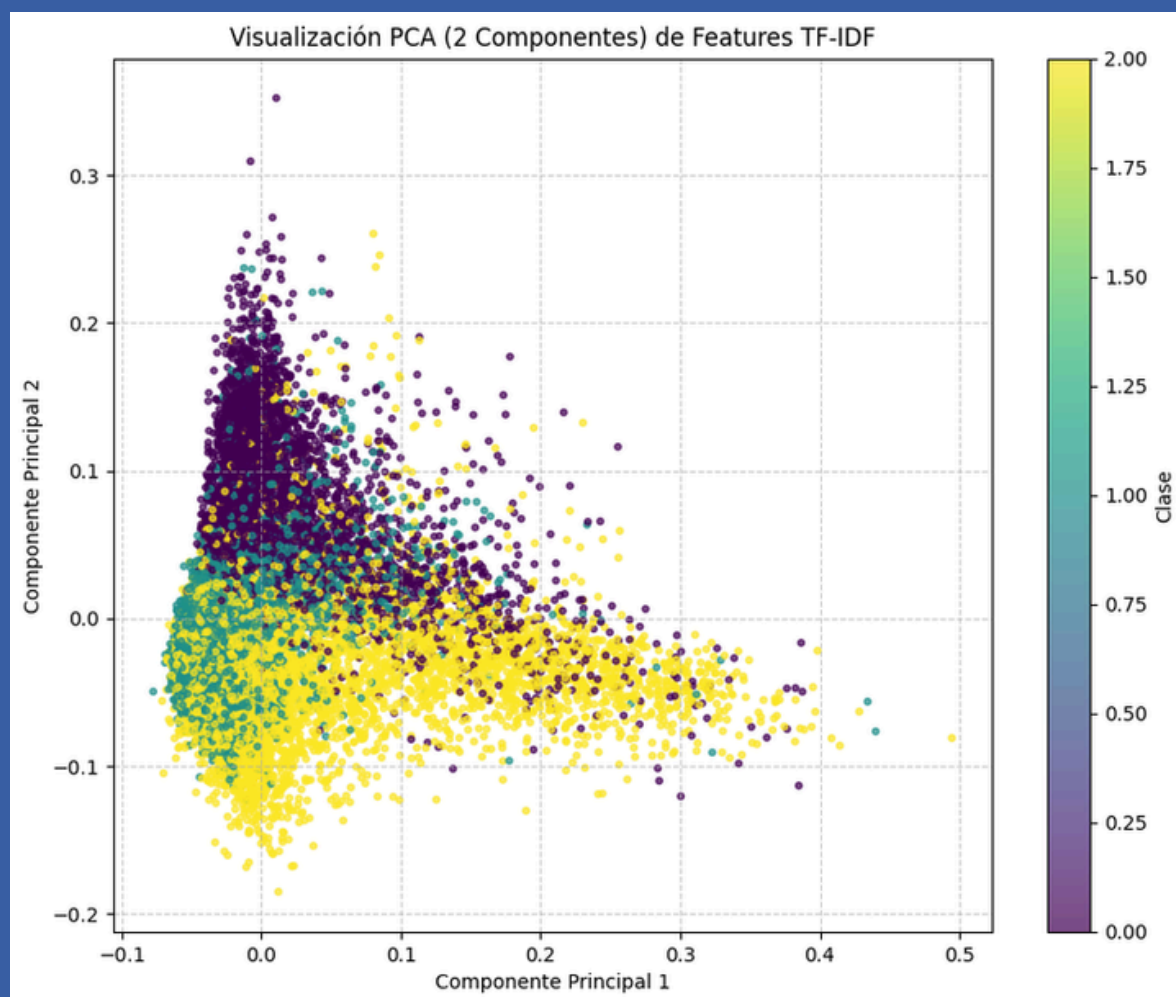
	Doc1_Index	Doc2_Index	Similarity
197361880	10009	14427	1.000000
223448373	11332	15329	1.000000
132465525	6718	6719	1.000000
302260844	15329	18951	1.000000
223451995	11332	18951	1.000000



Análisis de las features

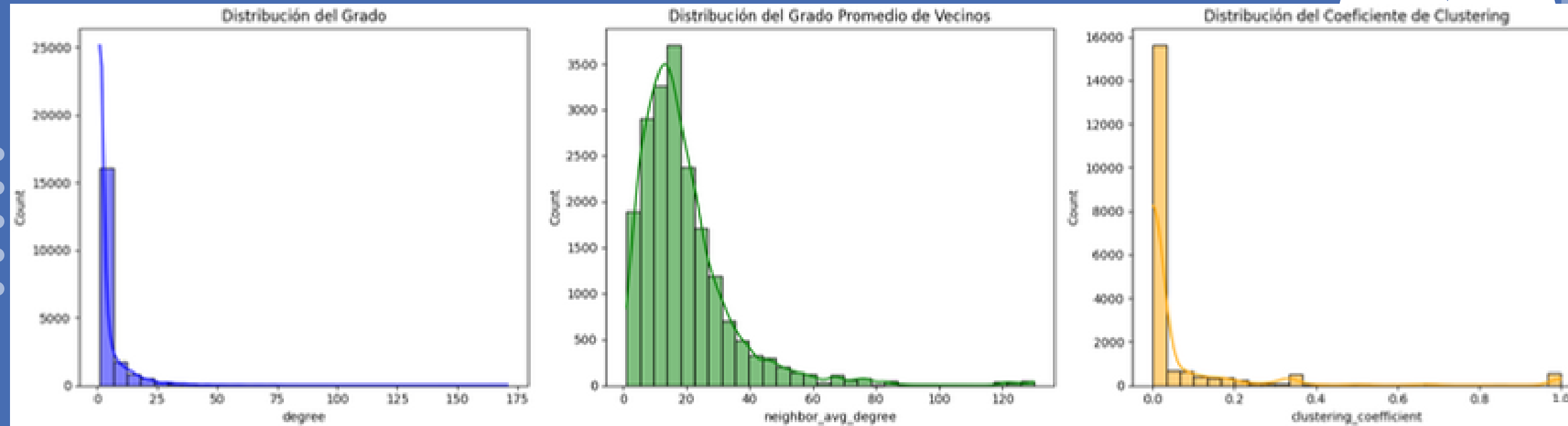
Indices con más relevancia en el TF-IDF:

- Term Index: 0, Total TF-IDF Weight: 344.6548
- Term Index: 196, Total TF-IDF Weight: 305.6691
- Term Index: 149, Total TF-IDF Weight: 302.1341
- Term Index: 89, Total TF-IDF Weight: 284.7277
- Term Index: 162, Total TF-IDF Weight: 278.5324



1. Hay pocos términos que parecen ser relevantes en muchos documentos
2. Al usar las features y realiza un clustering, pareciera ser que se ve una tendencia de división entre las 3 clases que estamos categorizando.

Métricas de integración



Grado de los nodos:

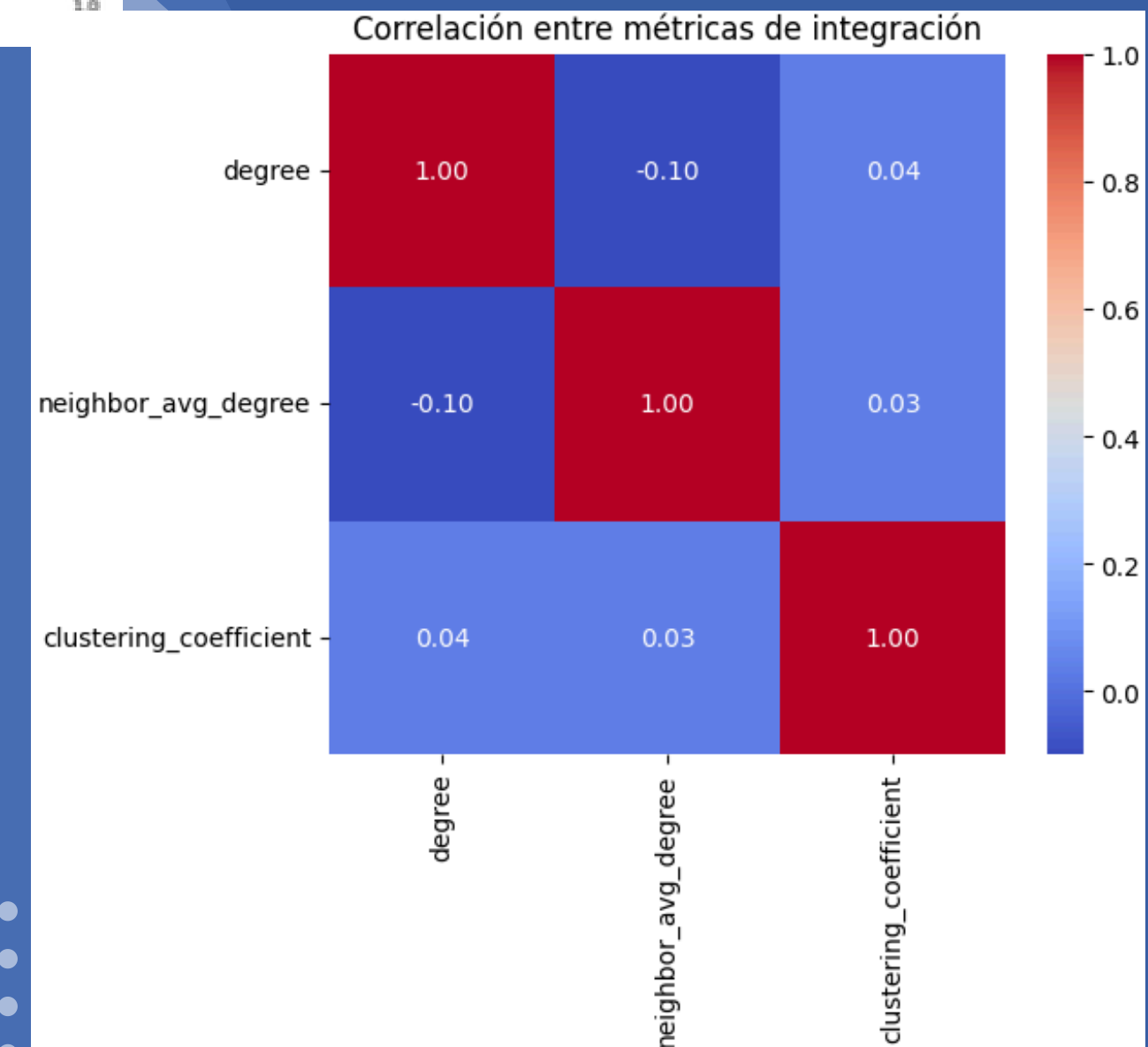
1. Media (4.50): Promedio de conexiones por nodo.
2. Mediana (2.0): 50% de los nodos tienen ≤ 2 conexiones.
3. Máximo (171): Nodo 11450 es un hub ultraconectado.

Grado promedio de vecinos:

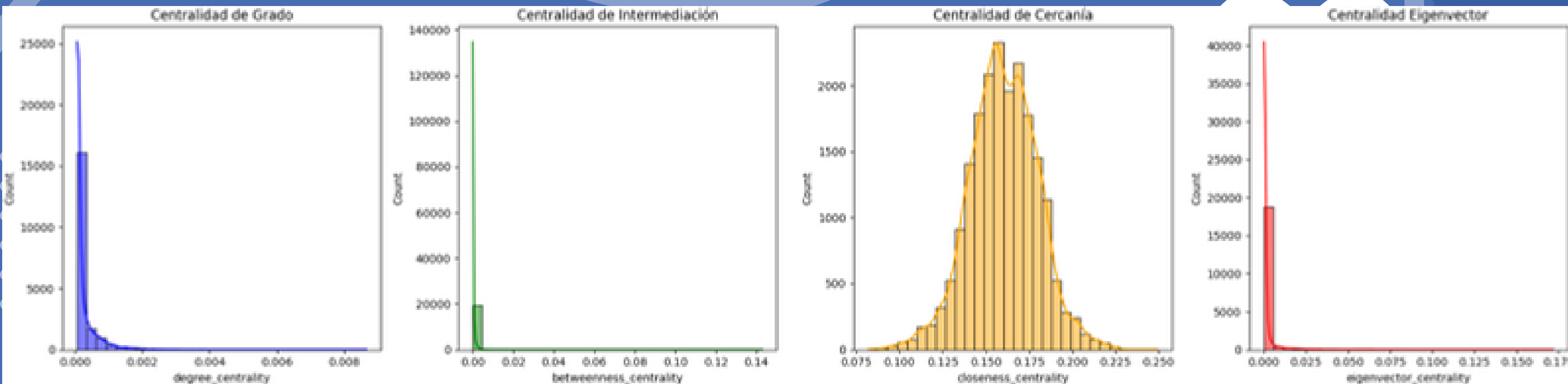
1. Media (19.27): Los vecinos de un nodo típico tienen ~ 19 conexiones.

Coeficiente de clustering:

1. 75% de nodos = 0.0: La mayoría no forma triángulos.

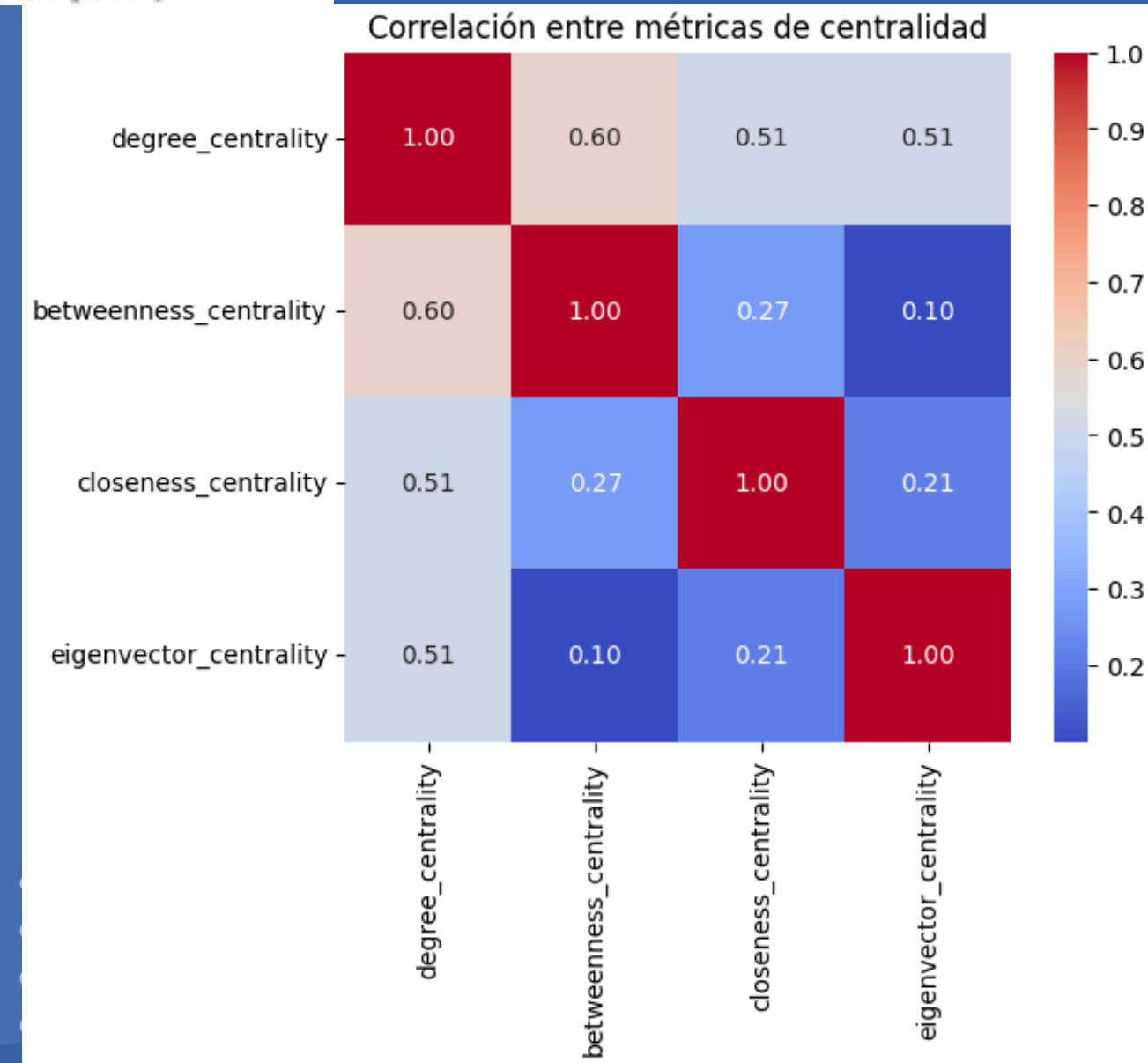


Métricas de Centralidad



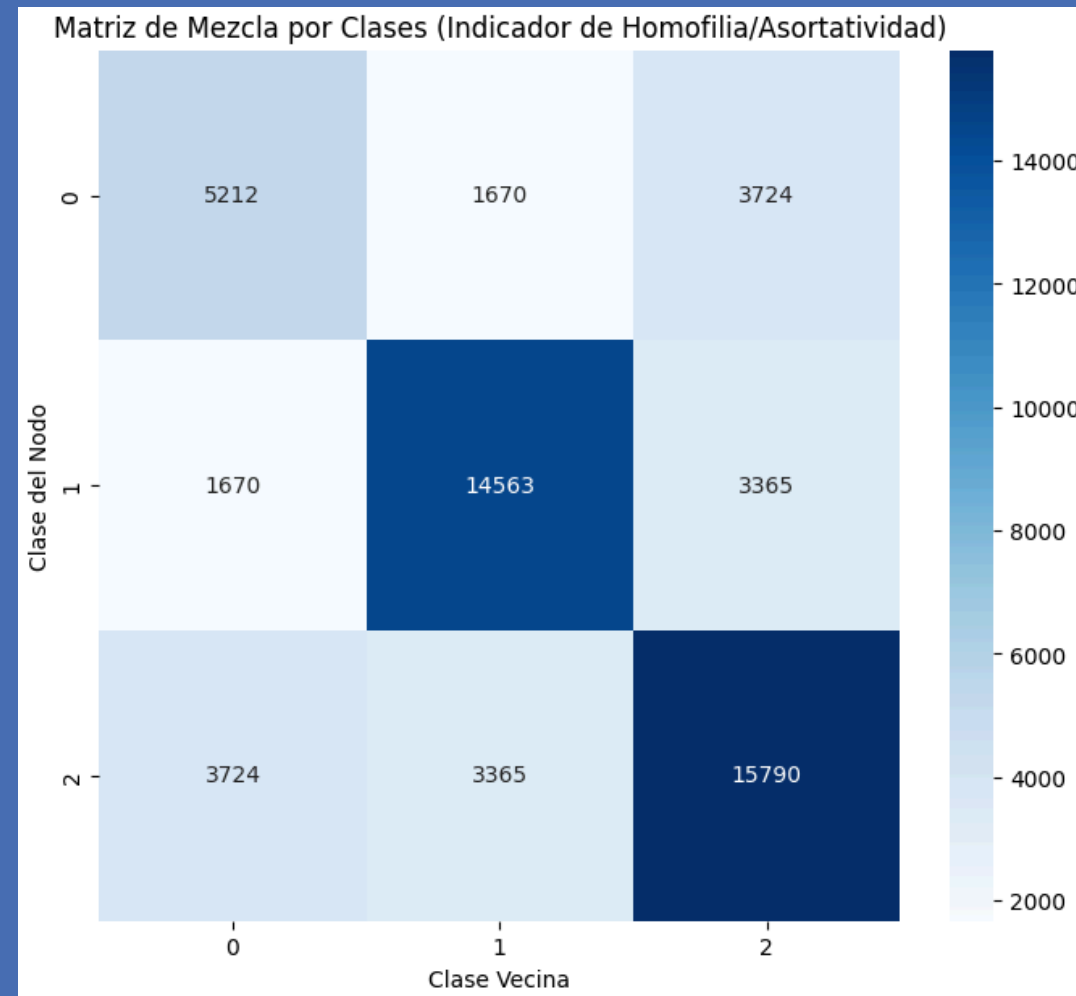
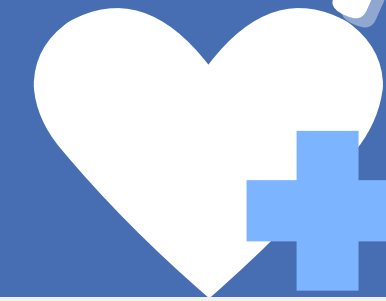
Análisis Global:

1. Red desigual: Pocos nodos concentran conexiones, intermediación e influencia (hubs).
2. Baja conectividad general: La mayoría de los nodos están poco conectados y no son críticos.
3. Posible estructura "scale-free": Presencia de nodos centrales y muchos periféricos (típico en redes como Internet o redes sociales).
4. Red libre de escala (scale-free) con hubs: Los hubs (alto degree) son también puentes (betweenness) y algo influyentes (eigenvector), pero no todos los puentes son influyentes.
5. Comunidades o módulos: La baja correlación entre betweenness y eigenvector podría indicar que la red tiene comunidades donde los conectores entre ellas no son los nodos más centrales dentro de cada grupo.

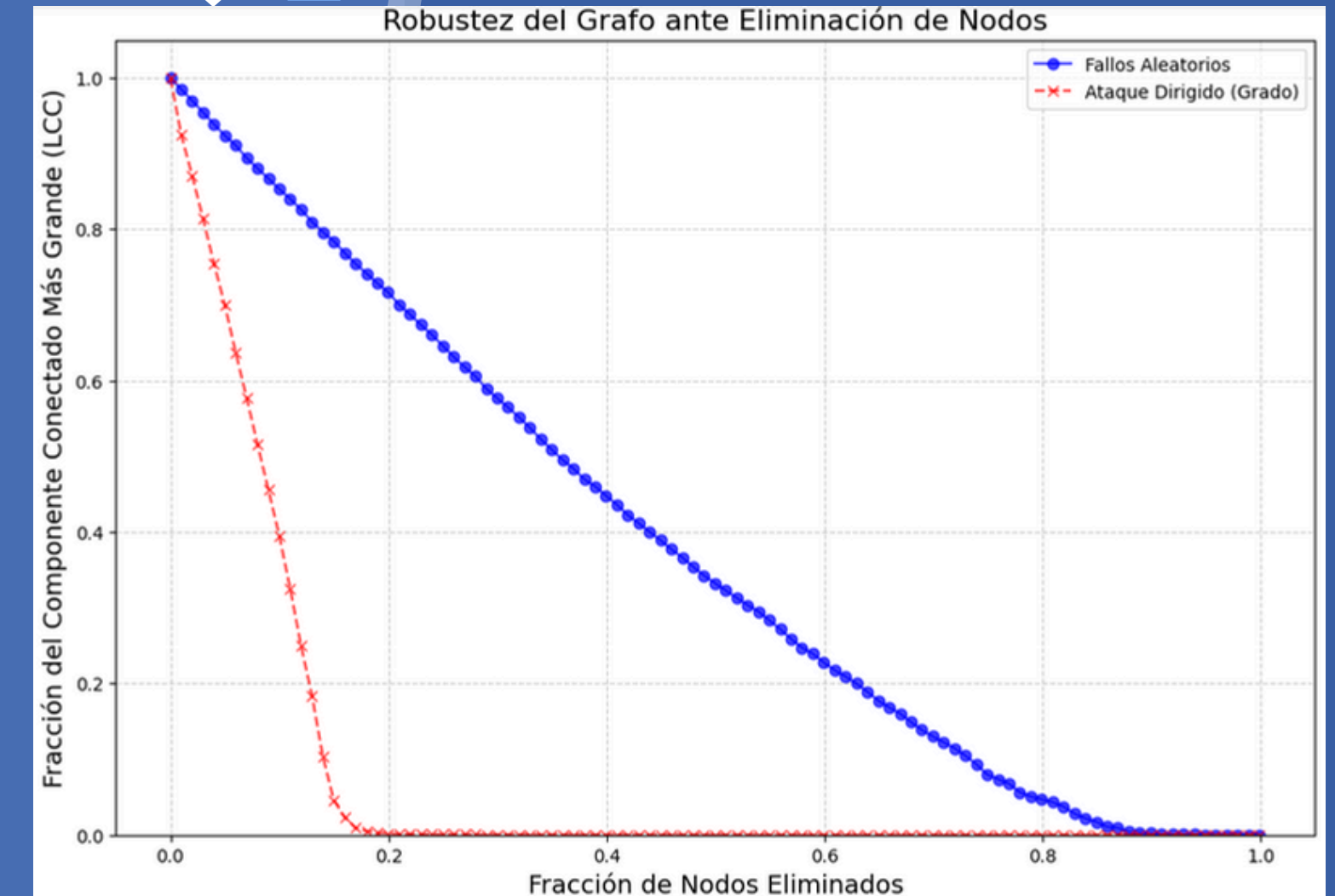


Métricas de Segregación y resiliencia

Coeficiente de Asortatividad
Global: 0.686



- Intra-clase (misma clase): 35,565 (80.24%).
- Inter-clase (clases distintas): 8,759 (19.76%).



- **Diámetro 18**
- **Camino más largo 171**

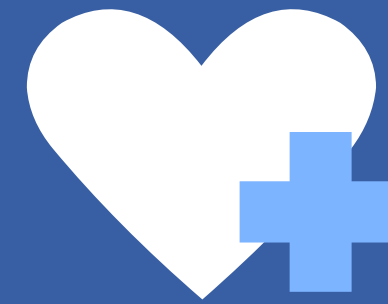
Preparación de los datos

1. Texto (Resúmenes médicos):

- Se usaron directamente las features vectorizadas con TF-IDF.
- No fue necesario preprocesamiento adicional.
- Se consideró aplicar reducción de dimensionalidad, pero:
 - Los modelos mostraron buen desempeño sin ella.
 - Se conservaron los vectores TF-IDF originales.

2. Grafo (Datos estructurados):

- Cada nodo tenía al menos 2 conexiones, asegurando conectividad.
- Se observó una distribución tipo Pareto:
 - 20% de los nodos concentraban el 80% de las conexiones.
- No se aplicaron transformaciones estructurales.
 -



Modelamiento

Modelo 1

Modelamientos
con solo
features

Modelo 3

Modelos con matriz
de adyacencia ,
Node2vec, features y
convolucional

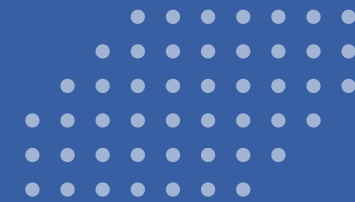
Modelo 2

Modelos con
Node2Vec

Modelo 4 y 5

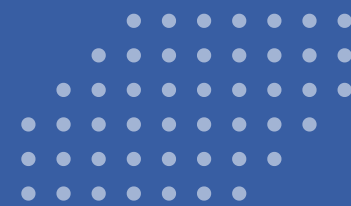
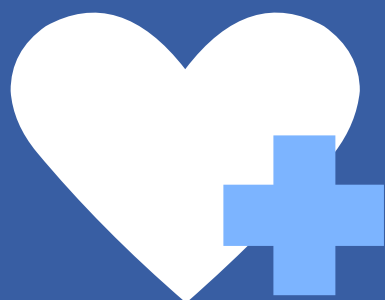
Modelos con
grados del nodo
y atención





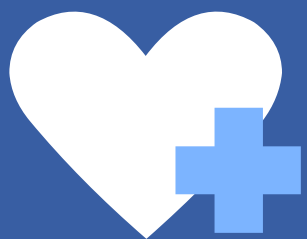
Modelo 1-Usando features

Regresión Logística	RandomForest	XGBoost-1	XGBoost-2	XGBoost-3	Features Estructurales
Max_iter=1000	n_estimators=100 random_state=42	n_estimators=100 learning_rate=0.1 max_depth=3	learning_rate=0.1, max_depth=3, n_estimators=200, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1, reg_lambda=0.1, random_state=42, tree_method='gpu_hist'	Mismos hiperparametros que el anterior pero con balanceo de clases.	
F1-Score=0.7303 Accuracy = 0.7290 AUC=0.8575	F1-Score=0.7563 Accuracy = 0.7390 AUC=0.8575	F1-Score=0.7128 Accuracy = 0.7080 AUC=0.8575	F1-Score=0.7111 Accuracy=0.7000 AUC=0.8575	F1-Score=0.7111 Accuracy=0.7000 AUC=0.8691	F1-Score=0.5279 Accuracy=0.5390 AUC=0.8575



Modelo 2 - Node2Vec

Modelo Node2Vec con regresión logística	Modelo Node2Vec con random forest	Modelo Node2Vec con xgboost	Modelo Node2Vec con red neuronal
- <i>One-vs-Rest (ovr)</i>, lo que significa que el modelo entrena un clasificador binario por cada clase, distinguiéndola del resto. - Embeddings de Node2Vec como variables predictoras (entradas) y las etiquetas reales de los documentos como salida - Función de pérdida: Log-Loss (pérdida logística multiclase)	- Búsqueda de hiperparámetros - Gridsearch - Mejor combinación: 'class_weight': 'balanced', 'max_depth': None, 'min_samples_leaf': 3, 'min_samples_split': 2, 'n_estimators': 200, 'random_state': 42 - Embeddings de Node2Vec como variables predictoras (entradas) y las etiquetas reales de los documentos como salida - Función de pérdida: Log-Loss (pérdida logística multiclase) - Optimizador Adam - Optimizador Adam	- Búsqueda de hiperparámetros - Gridsearch - Mejor combinación: 'colsample_bytree': 0.8, 'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 200, 'num_class': 3, 'objective': 'multi:softmax', 'random_state': 42, 'subsample': 0.8 - Embeddings de Node2Vec como variables predictoras (entradas) y las etiquetas reales de los documentos como salida - Función de pérdida: Log-Loss (pérdida logística multiclase) - Optimizador Adam - Optimizador Adam	- Búsqueda de hiperparámetros - Gridsearch - Mejor combinación: 'hidden_dim': 64, 'learning_rate': 0.001, 'dropout_rate': 0.2, 'num_layers': 3, 'epochs': 50 - Embeddings de Node2Vec como variables predictoras (entradas) y las etiquetas reales de los documentos como salida - Función de pérdida: Log-Loss (pérdida logística multiclase) - Optimizador Adam - Optimizador Adam
AUC train:0.9205 AUC test:0.9216 Acc Train:0.794 Acc Test: 0.803	AUC train:0.9991 AUC test:0.9394 Acc Train:0.978 Acc Test: 0.827	AUC train:1.00 AUC test:0.940 Acc Train:0.998 Acc Test: 0.834	AUC train:0.974 AUC test:0.942 Acc Train:0.887 Acc Test: 0.828

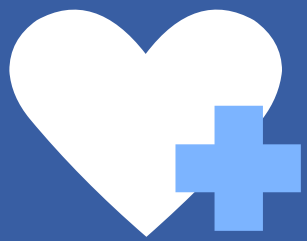


Modelo 3- Node2Vec + Matriz de Adyacencia

Modelo Node2Vec + matriz de adyacencia con red neuronal 1	Modelo Node2Vec + matriz de adyacencia con red neuronal 2
- Capa lineal y ReLU - Capa <i>dropout</i> , con el fin de reducir el riesgo de sobreajuste - Optimizador Adam - Función de pérdida de entropía cruzada - Entrenando el modelo en mini-lotes - Embeddings de Node2Vec como variables predictoras (entradas) junto con la matriz de adyacencia y las etiquetas reales de los documentos como salida - Regularización L2	- Búsqueda de hiperparámetros - Gridsearch - Mejor combinación: 'hidden_dim': 256, 'learning_rate': 0.01, 'dropout_rate': 0.7, 'weight_decay': 0.0001, 'epochs': 50 - Embeddings de Node2Vec como variables predictoras (entradas) junto con la matriz de adyacencia y las etiquetas reales de los documentos como salida - Función de pérdida: Log-Loss (pérdida logística multiclase) - Optimizador Adam - Regularización L2
AUC train:1.00 AUC test:0.916 Acc Train:0.999 Acc Test: 0.810	AUC train:0.997 AUC test:0.937 Acc Train:0.974 Acc Test: 0.826

◆ **Modelo 1 (Sin ajuste de hiperparámetros): Overfitting evidente, gran diferencia entre entrenamiento y prueba.**

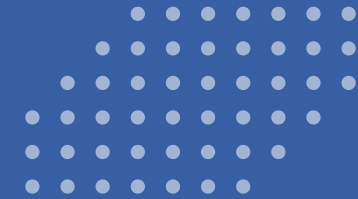
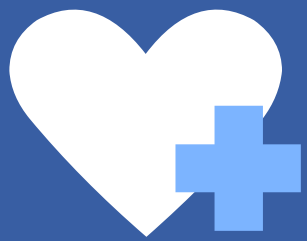
- ◆ **Modelo 2 (Con GridSearch) con Node2Vec + Adyacencia:**
- **Mejor equilibrio y capacidad de generalización.**
 - **Capturan tanto información vectorial como estructura del grafo.**
 - **Mejora significativa en la generalización y rendimiento.**



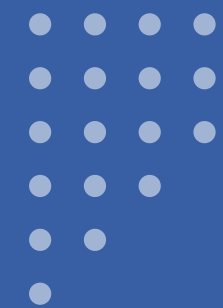
Modelo 3- Red Convolucional: Node2Vec + Matriz de Adyacencia + Features

Modelo Node2Vec + matriz de adyacencia+features con red neuronal convolucional
- Búsqueda de hiperparámetros
- Gridsearch
- Mejor combinación: {'hidden_dim': 64, 'learning_rate': 0.05, 'dropout_rate': 0.5, 'weight_decay': 1e-05, 'epochs': 100}
- Embeddings de Node2Vec como variables predictoras (entradas) junto con la matriz de adyacencia y features, al igual que las etiquetas reales de los documentos como salida
- Función de pérdida: Log-Loss (pérdida logística multiclase-Cross entropy)
- Optimizador Adam
- Regularización L2
AUC train:0.996
AUC test:0.971
Acc Train:0.959
Acc Test: 0.891

- No se detecta overfitting significativo:
 - Diferencias razonables entre train y test.
 - Uso de dropout y weight decay como regularización.
- Errores posibles:
 - En clases menos frecuentes o documentos ambiguos.
 - Común en clasificación multicategoría con datos reales.
- Ventaja en contexto PubMed:
 - Modelo robusto que combina texto + estructura del grafo.
 - Útil para tareas de curación automática, búsqueda y análisis bibliométrico.

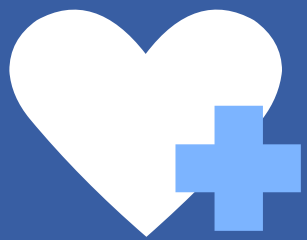


Modelo 4- Modelo con grados del nodo



Modelo 1
Entrada: grado normalizado del nodo (1 sola característica).
1 capa oculta (16 neuronas) → dropout → ReLU.
Capa de salida con tantas neuronas como clases.
Softmax logarítmico para clasificación multi-clase.
Puede mostrar que incluso características simples tienen poder predictivo en ciertos grafos.
Usa solo una feature sencilla (el grado del nodo) , sin embeddings, sin GCN, sin armar features manual

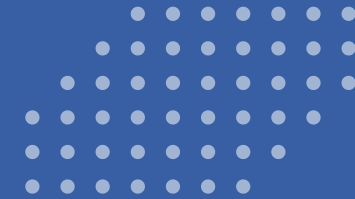
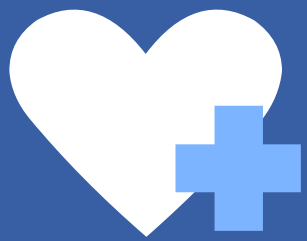
- Análisis del Modelo Basado en Grados
 - Rendimiento General
 - Exactitud (Accuracy): 31.5%
- Interpretación: El modelo solo acierta 1 de cada 3 predicciones, lo que sugiere que el grado de los nodos (número de conexiones) no es un buen predictor por sí solo para las clases objetivo.



Modelo 4- Modelo con grados del nodo y modelo con atención

Modelo 2
1 capa oculta con 32 neuronas.
Capa de salida con softmax logarítmico.
Dropout para regularización.
Es un modelo totalmente conectivo (no usa GCN todavía), pero más potente que el de grados simples.
Aprovecha información estructural del grafo , sin necesidad de usar GCN.
Es una excelente baseline basada en estadísticas de red .
Puede ser útil para comparar con GNNs más complejas (como GCN, GAT, GraphSAGE).
Combina PyTorch Geometric con networkx, lo cual es muy útil para análisis exploratorio estructural.

- Diagnóstico Detallado del Modelo de Clasificación
 - Rendimiento General (Accuracy: 31.4%)
- Interpretación crítica: El modelo tiene un rendimiento cercano al azar (33% sería aleatorio para 3 clases), indicando que no está aprendiendo patrones útiles con las features actuales.



Modelo 5- modelo con atención

Modelo 1

Simula paseos aleatorios (random walks) para capturar estructura local del grafo.

Entrena embeddings nodales sin supervisión.

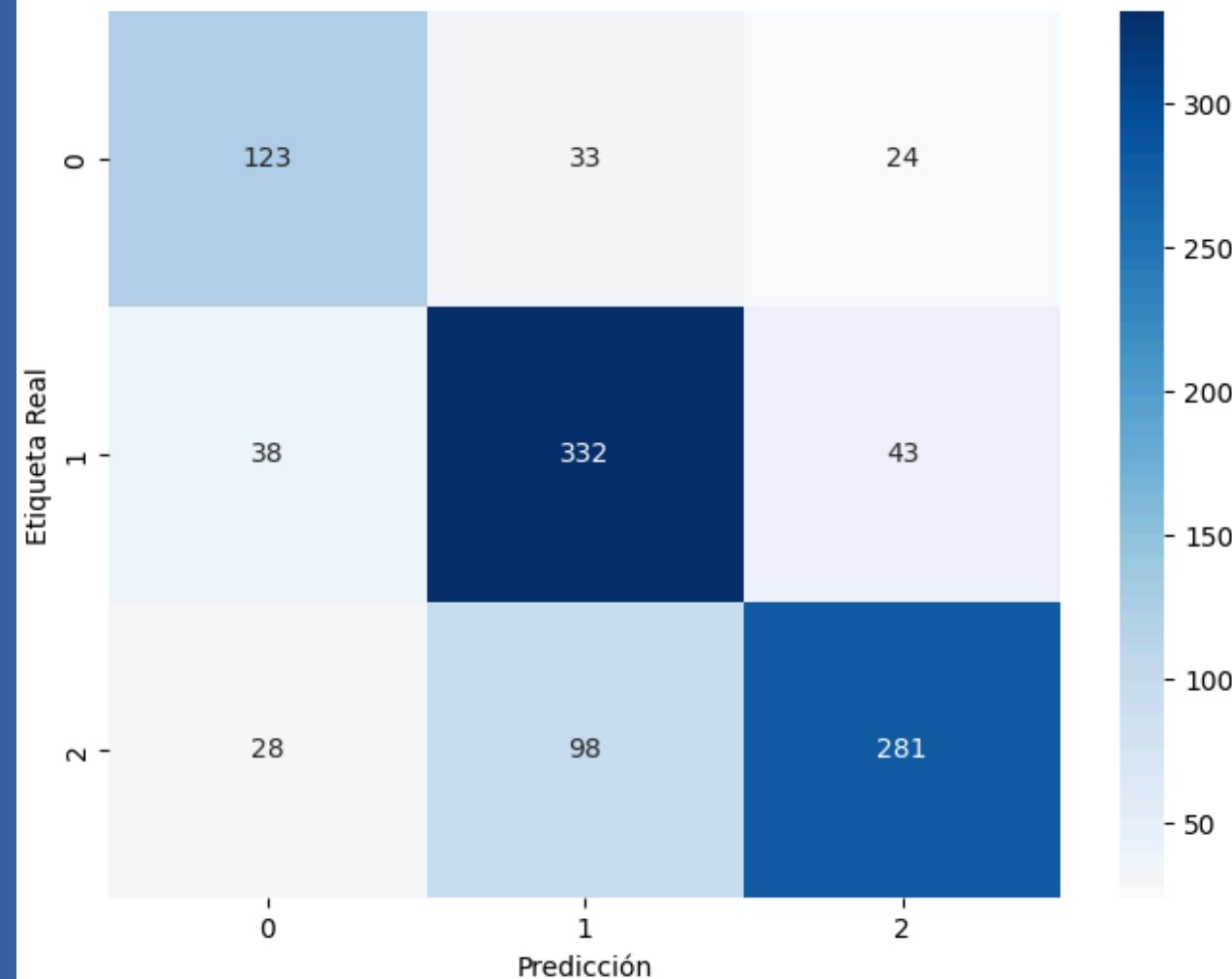
La red de atención usa GATConv para propagar la información entre nodos.

Separación de aprendizaje estructural (Node2Vec) y clasificadorio (GAT)

Uso de atención (GAT) permite que el modelo aprenda qué vecinos son más relevantes.

Node2Vec mejora la representación inicial, especialmente en grafos grandes o heterogéneos.

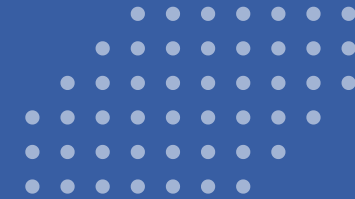
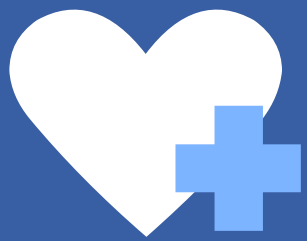
Matriz de Confusión - Test Set



- Análisis del modelo mejorado (Accuracy: 73.6%)

- Rendimiento General

- Exactitud del 73.6%: Mejora sustancial frente al 31.4% inicial.
- Balanceado: Buen equilibrio entre precisión y recall en las 3 clases.



Modelo 5- modelo con atención



Modelo 2

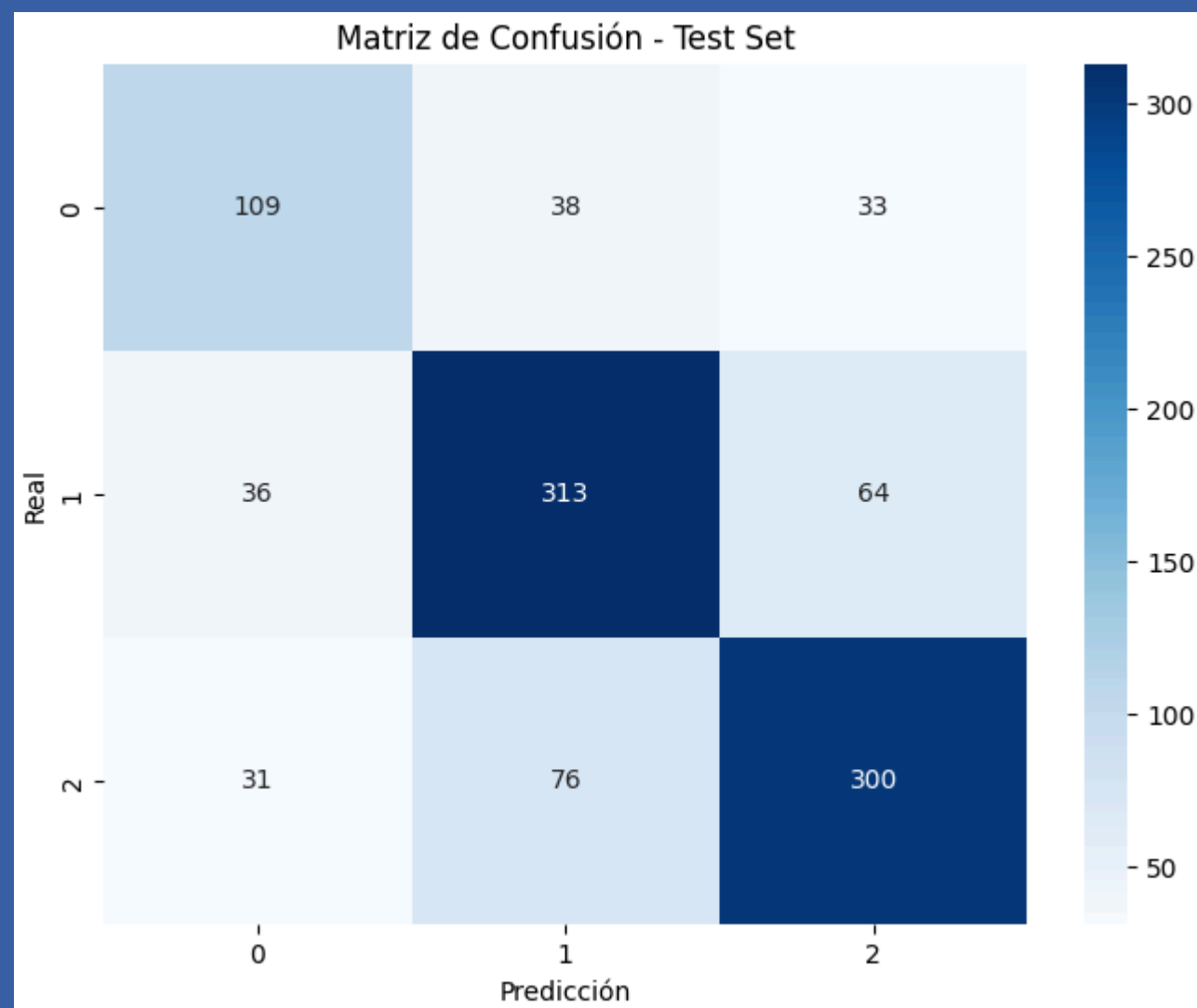
Mayor número de heads

BatchNorm1d añadido tras la primera capa de atención.

Embeddings de Node2Vec preentrenados

2 capas GATConv con más heads, BatchNorm, dropout

Preentrenamiento de Node2Vec + congelado (o fine-tune)



- Análisis del modelo final (Accuracy: 72.4%)
 - Rendimiento general
 - El modelo alcanza un 72.4% de exactitud, mostrando una mejora significativa respecto al baseline inicial (31.5%).
 - Esto indica que las modificaciones implementadas (nuevas features, balanceo, etc.) han sido efectivas.

Evaluación – Selección Propuesta

Progresión de Modelos

Modelos estructurales simples (ej. grado del nodo):

- Precisión baja (~31%)
- Insuficientes para clasificar correctamente.
- Node2Vec solo:
- Mejoran rendimiento (hasta 83.4% Accuracy con XGBoost)
- Problemas de overfitting.

Node2Vec + Matriz de Adyacencia (NN):

- Mejora en generalización
- Validan importancia de la conectividad explícita.

GAT + Node2Vec:

- Salto cualitativo
- Manejo efectivo de clases difíciles (Clase 1)
- Accuracy: 72.4–73.6%

Modelo Final (TripleInputModel)

Combinación:

- Node2Vec + Matriz de Adyacencia + TF-IDF + CNN
- Precisión (test): 89.1%
- AUC (test): 0.971
- Ventaja: Fusión sinérgica de estructura y semántica
- Arquitectura optimizada con técnicas de regularización

Impacto Potencial

- Automatización fiable y precisa de la categorización documental
- Ahorro de tiempo y reducción de errores en curación
- Mejora en la recuperación de información médica
- Base sólida para análisis bibliométricos y nuevos servicios

Conclusiones



🧠 Conclusiones del Proyecto

- Se desarrolló un sistema de clasificación automática de documentos médicos para PubMed usando PyTorch.
- Se abordó un problema multicategoría complejo, crucial para la Biblioteca Nacional de Medicina de EE. UU.
- Métricas clave:
 - Accuracy: mide la precisión general de clasificación.
 - AUC: evalúa la capacidad de discriminación entre clases, especialmente útil ante clases desbalanceadas.
- El mejor modelo alcanzó:
 - 89.1% Accuracy
 - 0.971 AUC
 - Alta generalización y baja propensión al overfitting.

Conclusiones

Implicaciones para el Negocio

-  **Eficiencia Operativa:**
 - Automatiza la clasificación documental
 - Reduce errores y tiempos
 - Libera recursos humanos para tareas especializadas
-  **Mejora en Recuperación de Información:**
 - Búsquedas más precisas, rápidas y confiables
 - Impacto directo en investigación y salud pública
-  **Consistencia y Calidad de Datos:**
 - Clasificación uniforme
 - Facilita análisis bibliométricos más sólidos
-  **Nuevas Capacidades Analíticas:**
 - Detección de tendencias emergentes
 - Identificación de vacíos de conocimiento
 - PubMed como referente global en tecnología aplicada a bibliotecas médicas



GRACIAS

